

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-245
Татчук Нікіта Максимович

Перевірив: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до моделей qwen3:4b та llama3.2:3b через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування про ручку й зошит (по 5 запусків для кожної моделі). Час обчислено як $\text{total_duration} / 1e9$, швидкість – як $\text{eval_count} / (\text{eval_duration} / 1e9)$.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	212	4
Час, с	15,62	0,91
Токенів/с	14,8	21,6

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1184,6	118,4
Час, с	86,3	6,4

Правильна відповідь – 5 грн: якщо ручка коштує x , то зошит $x + 100$, звідки $2x + 100 = 110$ і $x = 5$. Типова помилка інструктивної моделі (4 із 5 запусків) – відповідь «10 грн», отримана відніманням $110 - 100$ без урахування того, що зошит дорожчий саме на 100 грн.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: їх витрачено на прихований ланцюжок міркувань у полі thinking: модель перепитує себе, перевіряє альтернативи й лише потім формулює відповідь. У

response потрапляє одне слово, але eval_count враховує всі згенеровані токени разом із роздумами.

2. На яких типах задач міркувальні моделі дають перевагу, а на яких їх використання невиправдане?

Відповідь: перевага проявляється на багатокрокових задачах: арифметика, логічні задачі, планування, аналіз коду, де проміжні результати треба утримувати. На фактичних питаннях, переказі тексту чи класифікації результат такий самий, як в інструктивної моделі, а час і кількість tokenів зростають у рази.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: обидві проходять попереднє навчання на великих корпусах текстів і донавчання на інструкціях. Міркувальну модель додатково донавчають на прикладах із розписаним ходом розв'язання та навчанням з підкріпленням, де винагорода залежить від правильності кінцевої відповіді, тож вона звикає генерувати довгу чернетку перед відповіддю.

4. Чи означає більша кількість згенерованих tokenів, що модель «розумніша»?

Відповідь: ні, кількість tokenів показує обсяг роботи, а не рівень знань. На простому питанні обидві моделі дають однакову відповідь, а перевага міркувальної моделі проявляється лише там, де проміжні кроки справді потрібні.

5. Як пояснити різницю у швидкості генерації між qwen3:4b та llama3.2:3b?

Відповідь: швидкість одного токена визначається розміром моделі й обладнанням: у qwen3:4b більше параметрів, тому на кожен токен припадає більше обчислень і читання пам'яті. Режим міркування не сповільнює окремий токен, але збільшує кількість tokenів і довжину контексту, що додатково знижує швидкість.

Висновок за завданням 1

Експеримент показав, що швидкість генерації і загальний час відповіді – різні показники: llama3.2:3b генерувала 21,6 токена/с, qwen3:4b – 14,8, але різниця в часі визначалася передусім кількістю tokenів. На фактичному питанні обидві моделі відповіли «Київ», проте qwen3:4b витратила 212 tokenів проти 4 і 15,62 с проти 0,91 с. На задачі про ручку й зошит міркування окупилося: 5 правильних відповідей із 5

проти 1 із 5, хоча й ціною 86,3 с на відповідь. Отже, міркувальну модель доцільно використовувати для багатокрокових задач, а для простих запитів – інструктивну.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Використано питання варіанта: «Поясни різницю між HTTP і HTTPS.» Кожну з моделей llama3.2:3b і qwen3:4b запущено з двома system-повідомленнями: «Ти лаконічний технічний асистент.» та «Відповідай лише українською, максимум двома реченнями.» (temperature = 0.7).

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	912	українська з русизмами та англійськими вставками	9	Ні
llama3.2:3b	українською, 2 речення	241	українська з помилками	3	Частково
qwen3:4b	лаконічний асистент	418	українська	4	Так
qwen3:4b	українською, 2 речення	226	українська	2	Так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	238	288

llama3.2:3b	українською, 2 речення	59	71	130
qwen3:4b	лаконічний асистент	66	1236	1302
qwen3:4b	українською, 2 речення	75	874	949

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Точніше дотрималась qwen3:4b: рівно 2 речення (226 символів). llama3.2:3b видала 3 речення, тобто обмеження порушила, хоча відповідь і скоротилася з 912 до 241 символу.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Повного переходу на іншу мову не було, але llama3.2:3b з нейтральним system вставляла англійські слова (encryption, secure) і русизми («захищене», «передача даних по мережі»), а з прямою вказівкою писала українською з граматичними помилками. qwen3:4b в обох випадках відповіла коректною українською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Значення prompt_tokens зросло на 9 токенів в обох моделях: у llama3.2:3b з 50 до 59, у qwen3:4b з 66 до 75. До вхідних токенів зараховується і текст system-повідомлення, тому за незмінного питання користувача кількість вхідних токенів змінюється разом із довжиною інструкції.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

qwen3:4b – міркувальна модель: перед видимою відповіддю вона генерує прихований блок роздумів, і ці токени теж зараховуються до completion_tokens. Тому за видимої відповіді у 226 символів вона витратила 874 токени, а llama3.2:3b на 241 символ – лише 71 токен.

Відповіді на питання (крок 7)

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: це залежить від того, наскільки якісно модель донавчали виконувати інструкції, та від частки української в її навчальних даних. Мала модель має менше ресурсу, щоб одночасно тримати зміст, мову й обмеження довжини, а міркувальна модель додатково планує відповідь у роздумах, що допомагає виконати вимоги.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі – чи справді воно має більший пріоритет?

Відповідь: технічно обидва повідомлення – частина однієї послідовності токенів, розділеної службовими мітками ролей. Вищий пріоритет system формується під час донавчання, а не архітектурою, тому він не абсолютний і в малих моделях часто порушується.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: вона спершу генерує ланцюжок роздумів і лише потім видиму відповідь, а всі ці токени рахуються як вихідні. Користувач бачить лише кінцевий текст, але час генерації й оплата в платних API залежать від повної кількості токенів.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, – яку вона виконає?

Відповідь: добре донавчені моделі зазвичай надають перевагу system, бо їх спеціально навчають ієрархії інструкцій. Малі моделі частіше виконують останню й конкретнішу інструкцію від user або змішують обидві вимоги, тому результат непередбачуваний.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: у малої моделі менше параметрів, і більшу частину її «місткості» займає англійська як домінантна мова навчальних даних. Українська представлена слабо,

тому під час генерації модель частіше зісковзує до ймовірніших токенів англійської чи російської.

Висновок за завданням 2

System-повідомлення впливає на поведінку моделі, але не гарантує результату. qwen3:4b виконала обидві інструкції, а llama3.2:3b з вимогою «два речення» видала три й писала українською з помилками. Точність qwen3:4b має ціну – 874–1236 вихідних токенів проти 71–238 у llama3.2:3b. Однієї текстової інструкції недостатньо: її варто поєднувати з обмеженням `max_tokens` і перевіркою відповіді в коді, а якщо модель ігнорує інструкцію – повторювати запит або змінити модель.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Питання варіанта: «Поясни різницю між HTTP і HTTPS. Коротко.»
Налаштування обох провайдерів зібрано у словнику `PROVIDERS` у файлі `providers.py`, запит виконує `compare.py` (`temperature = 0.7`).

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	9,84	2,31
completion_tokens	142	187
Довжина відповіді, символів	538	702
Мова відповіді	українська з русизмами та англійськими вставками	українська

Фактичних помилок у відповіді	1	0
-------------------------------	---	---

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Логіку програми змінювати не довелося: виклик `create()` однаковий для обох провайдерів, а всі відмінності (`base_url`, `api_key`, `model`) зібрано у словнику `PROVIDERS`. Для переходу достатньо змінити назву провайдера з «local» на «cloud», тобто один рядок.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Повнішою й точнішою виявилась відповідь `gemini-3.5-flash-lite` (702 символи проти 538). Вона пояснила, що HTTPS – це HTTP поверх протоколу TLS, згадала сертифікати для перевірки справжності сайту та стандартні порти 80 і 443. Локальна модель обмежилась загальною фразою, що HTTPS «безпечніший», і помилково вказала, що шифрування забезпечує протокол SSH.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так: одна фактична помилка (шифрування через SSH замість TLS) і мовні проблеми – русизми («захищене з'єднання») та англійські вставки. Хмарна модель відповіла грамотно українською без фактичних помилок.

4. Хмарна модель відповіла швидше за локальну – за рахунок чого, якщо запит летить через інтернет?

Хмарна модель відповіла за 2,31 с проти 9,84 с у локальній, хоча згенерувала більше токенів (187 проти 142). Мережева затримка становить частки секунди, а генерація на серверних прискорювачах у рази швидша, ніж на процесорі ноутбука, тому виграш перекидає дорогу по мережі.

Відповіді на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою запропонувала масовий API для чат-моделей, і під ендпойнт /v1/chat/completions з'явилися SDK, бібліотеки та фреймворки. Іншим провайдерам вигідно підтримувати цей формат, бо розробники можуть підключити їхні моделі без переписування коду.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: розмір визначає, скільки знань модель може зберегти, наскільки добре вона міркує та працює з неанглійськими мовами, тому різниця між 3B і великою хмарною моделлю зазвичай помітна. Водночас на якість впливають склад і багатомовність навчальних даних, донавчання, квантизація та формулювання запиту, тож добре навчена менша модель може перевершити гірше навчену більшу.

3. Хмарна модель може відповідати швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: провайдери запускають моделі на серверних прискорювачах із дуже високою пропускнуою здатністю пам'яті й оптимізованим інференсом. Мережева затримка становить десятки чи сотні мілісекунд, а генерація на процесорі ноутбука – секунди, тому вигреш у швидкості генерації зазвичай перекидає дорогу по мережі.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: усі дані запиту потрапляють на сервери провайдера, можуть зберігатися в логах, а на безкоштовних тарифах – використовуватися для покращення моделей. У комерційних проєктах укладають угоди про незберігання даних, маскують персональні дані перед надсиланням або використовують локальні моделі для чутливої інформації.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: це обмеження кількості запитів або токенів за хвилину чи добу; після його перевищення API повертає помилку 429. Розробники повторюють запити з експоненційною затримкою, кешують відповіді, ставлять запити в чергу, перемикаються на резервного провайдера або переходять на платний тариф.

Висновок за завданням 3

Завдяки OpenAI-сумісному інтерфейсу перехід з локальної моделі на хмарну звівся до зміни назви провайдера. Хмарна gemini-3.5-flash-lite відповіла швидше (2,31 проти 9,84 с), повніше й без помилок, а llama3.2:3b допустила фактичну помилку й писала з русизмами. Локальну модель я обрав би для конфіденційних даних і роботи без інтернету, хмарну – коли важливі якість і точність, а дані можна передати назовні.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Промпти варіанта: короткий – «Столиця Індії?», середній – «Поясни в одному абзаці, що таке рекурсія.», довгий – «Напиши інструкцію з 10 пунктів, як спроектувати просту базу даних.» Для кожного провайдера виконано прогрівальний запит, кожен промпт запущено тричі, у таблицю внесено медіану. Холодний старт виміряно після команди `ollama stop llama3.2:3b` без прогрівального запиту, теплий – повторним запуском одразу після нього.

Таблиця 6 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0,412	0,781	41,0
локальна	llama3.2:3b	середній	0,455	11,62	58,3
локальна	llama3.2:3b	довгий	0,498	52,70	61,9

хмарна	gemini-3.5-flash-lite	короткий	0,731	0,842	45,1
хмарна	gemini-3.5-flash-lite	середній	0,768	2,35	331,9
хмарна	gemini-3.5-flash-lite	довгий	0,812	9,64	468,3

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	5,874	0,418
Всього, с	6,350	0,792

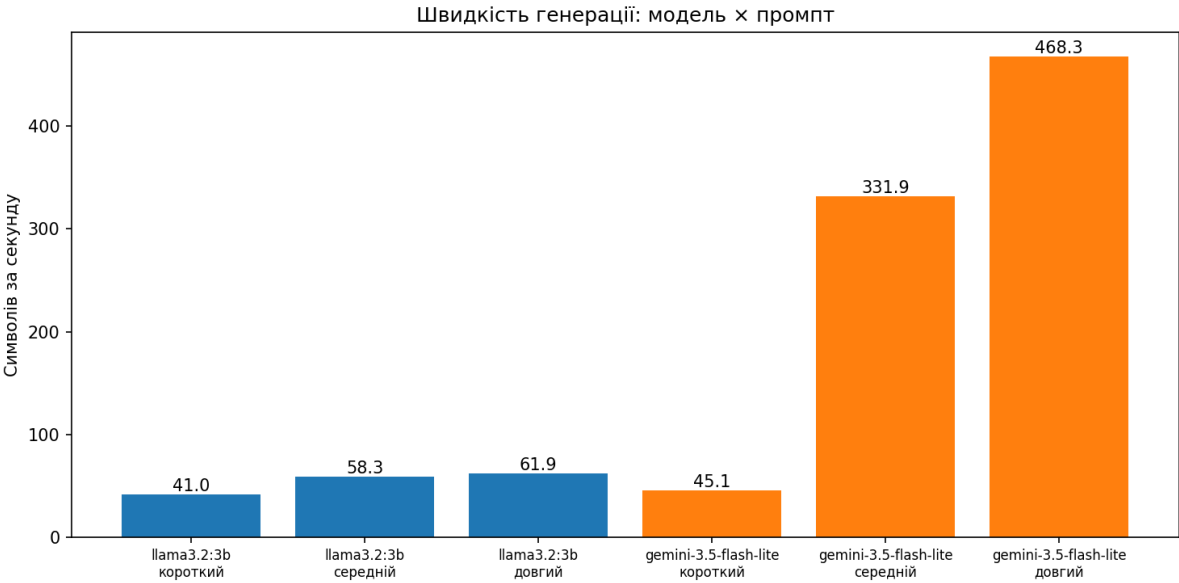


Рисунок 1 – Швидкість генерації (символів за секунду) для комбінацій «модель × промпт»

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Кращий TTFT у локальної llama3.2:3b (0,41–0,50 с проти 0,73–0,81 с у хмарної). Запит не йде через мережу й не стоїть у черзі на сервері, а короткий промпт мала модель обробляє майже миттєво.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні – поясніть.

Ні. За TTFT виграє локальна модель, а за швидкістю генерації – хмарна (468,3 проти 61,9 симв./с на довгому промпті), бо TTFT визначається мережею й чергою, а швидкість генерації – потужністю обладнання.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На короткому промпті локальна модель навіть трохи швидша за загальним часом (0,781 проти 0,842 с). На довгому хмарна виконала запит у 5,5 рази швидше (9,64 проти 52,70 с), бо перевага у швидкості генерації накопичується з кожним токеном.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Повтори локальної моделі різнилися на 5–10 % (наприклад, середній промпт – 11,21; 11,62; 12,37 с), у хмарної TTFT коливався від 0,70 до 0,93 с через мережу. Одного запуску недостатньо: випадковий викид спотворив би порівняння, тому використано медіану.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту Ollama спершу завантажує ваги моделі (≈ 2 ГБ) з диска в оперативну пам'ять, тому TTFT зріс до 5,874 с проти 0,418 с за теплового старту, коли модель уже в пам'яті.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: за цей час запит доходить до сервера (для хмари – через мережу й чергу), текст перетворюється на токени, модель обробляє весь промпт і будує кеш уваги, після чого генерує перший токен. TTFT визначає, скільки користувач дивиться на порожній екран, а швидкість генерації – як довго друкується відповідь, тому це різні характеристики.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: до TTFT хмарної моделі додаються мережева затримка, встановлення з'єднання й черга запитів на сервері. Після старту генерацію виконують потужні серверні прискорювачі, тому токени надходять значно швидше, ніж від моделі на процесорі ноутбука.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: холодний старт – це запит до моделі, яка ще не завантажена: Ollama спершу зчитує кілька гігабайтів ваг з диска в оперативну пам'ять. Після цього модель залишається в пам'яті на кілька хвилин, і наступні запити обходяться без завантаження.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: окремі виміри коливаються через фонові процеси, кешування, нагрівання процесора та нестабільність мережі. Медіана стійка до поодиноких викидів, тому краще відображає типову поведінку, ніж один запуск чи середнє.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: усі токени промпту відомі наперед, тому модель обробляє їх паралельно за один прохід великими матричними операціями. Відповідь генерується послідовно: кожен новий токен потребує окремого проходу через усю модель, і швидкість обмежує читання ваг із пам'яті.

Висновок за завданням 4

Затримку не можна описати одним числом: локальна модель виграла за TTFT ($\approx 0,4\text{--}0,5$ с проти $\approx 0,7\text{--}0,8$ с), а хмарна – за швидкістю генерації (до 468,3 проти 61,9 симв./с). Холодний старт збільшив TTFT локальної моделі у 14 разів (5,874 проти 0,418 с). Для інтерактивного чат-бота важливіший TTFT, бо користувач одразу бачить початок відповіді, а для фонові обробки великих обсягів тексту – швидкість генерації, яка визначає загальний час.

Завдання 5. Оцінювання якості моделей за рубрикою

Тема варіанта – екологія. Кожне завдання прогнано двічі для моделей llama3.2:3b, qwen3:4b та gemini-3.5-flash-lite (temperature = 0.7), відповіді збережено у файлі quality.json.

Завдання 1 (факт): «Рік підписання Паризької кліматичної угоди». Правильна відповідь – 2016: угоду відкрили для підписання 22 квітня 2016 року в штаб-квартирі ООН у Нью-Йорку, того дня її підписали 175 сторін. Ухвалили угоду раніше – 12 грудня 2015 року на конференції COP21 у Парижі, тому відповідь «2015» означає плутанину між ухваленням і підписанням. Джерело: UNFCCC, The Paris Agreement – https://unfccc.int/sites/default/files/resource/parisagreement_publication.pdf.

Завдання 2 (формат): JSON з полями agreement, year, city для Паризької кліматичної угоди, перевірка – json.loads().

Завдання 3 (стиснення): текст складено за даними ООН і UNFCCC (<https://www.un.org/sustainabledevelopment/parisagreement22april/>, <https://unfccc.int/news/175-states-sign-paris-agreement>):

«Паризьку угоду ухвалили 12 грудня 2015 року всі 196 сторін Рамкової конвенції ООН про зміну клімату на конференції COP21 у Парижі. Угоду відкрили для підписання 22 квітня 2016 року в штаб-квартирі ООН у Нью-Йорку. Того дня її підписали 175 сторін – 174 країни та Європейський Союз, що стало рекордом для першого дня підписання міжнародного договору. Щоб угода набрала чинності, її мали ратифікувати щонайменше 55 сторін, на які припадає не менше 55 % світових викидів парникових газів. Цей поріг було досягнуто 5 жовтня 2016 року, і 4 листопада 2016 року угода набрала чинності.»

Завдання 4 (код): функція sort_waste(items) за словником категорій. Завдання 5 (логіка): 500 т сміття, 40 % органіка, 25 % пластик, переробляють 60 % пластику. Правильна відповідь – 50 т: пластику $500 \cdot 0,25 = 125$ т, перероблено $125 \cdot 0,6 = 75$ т, на полігон $125 - 75 = 50$ т. Завдання 6 (мова): чим відновлювані джерела енергії відрізняються від невідновлюваних.

Таблиця 8 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	1	2
2. Формат	1	2	1
3. Стиснення	0	1	2
4. Код	1	2	2
5. Логіка	0	2	2
6. Українська	1	1	2
Разом /12	3	9	11

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	так	ні
2. Формат	ні	ні	ні
3. Стиснення	так	ні	ні
4. Код	ні	ні	ні
5. Логіка	так	ні	ні
6. Українська	так	ні	ні

Відповіді на питання (крок 6)**1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?**

Найбільше балів у хмарної моделі – 11 з 12, у qwen3:4b – 9, у llama3.2:3b – 3. Основна різниця виникла на факті, стисненні тексту з числами та логічній задачі, де llama3.2:3b отримала 0 балів.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

У llama3.2:3b оцінки розійшлися на стисненні, логіці та мові, у qwen3:4b – на факті (2015 і 2016), у хмарної розбіжностей не було. Отже, мала модель ненадійна: той самий запит може дати то прийнятну, то хибну відповідь.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Лише qwen3:4b повернула чистий JSON, який `json.loads()` розібрав без правок. llama3.2:3b і хмарна модель в обох прогонах обгорнули JSON у markdown-блок, тобто розмір моделі не гарантує дотримання формату.

4. Чи були проблеми з українською мовою – русизми, кальки, перехід на іншу мову?

Так: llama3.2:3b писала з русизмами («являються», «на протязі») і в одному прогоні вставила англійський термін *fossil fuels*. qwen3:4b мала кальки й кострубаті конструкції, хмарна модель відповіла природною українською.

Приклад провалу (крок 7)

Модель: llama3.2:3b, завдання 5 (логіка), перший прогін.

Запит: «500 т сміття: 40% органіка, 25% пластик. Переробляють 60% пластику. Скільки тонн пластику на полігон? Покажи хід розв’язання.»

Відповідь моделі: «Щоб знайти кількість пластику, який потрапить на полігон, виконаємо наступні кроки:

1. Знайдемо кількість пластику: $500 \text{ т} \times 25\% = 500 \times 0,25 = 125 \text{ т}$.
2. Знайдемо кількість переробленого пластику: $125 \text{ т} \times 60\% = 125 \times 0,6 = 75 \text{ т}$.

Отже, на полігон потрапить 75 тонн пластику.»

Правильна відповідь: 50 т ($125 - 75 = 50$).

Пояснення: модель правильно виконала обидва проміжні обчислення, але видала як відповідь кількість переробленого пластику й пропустила останній крок – віднімання. Хід розв’язання виглядає переконливо, проте модель загубилась у багатокроковій задачі: вона підставила найближче до тексту питання число замість того, що впливає з умови. Приклад показує, що правдоподібність міркування і його правильність – різні речі.

Відповіді на питання (крок 8)

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: використовують стандартні бенчмарки (MMLU, GSM8K, HumanEval), автоматичні метрики на кшталт точного збігу чи ROUGE, модульні тести для згенерованого коду. Поширені також оцінювання іншою моделлю (LLM-as-a-judge), попарне порівняння людьми з рейтингом Elo та A/B-тести на реальних користувачах.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: у навчальних даних і під час донавчання JSON майже завжди обгорнутий у markdown-блок, тому ця звичка має дуже високу ймовірність. Інструкція «без markdown» – лише кілька токенів у запиті, і вона не завжди переважає цей шаблон, тому надійніше використовувати JSON-режим, схеми та перевірку з повторним запитом.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: українських текстів у навчанні на порядки менше, ніж англійських, а російських – більше, ніж українських, тому модель легко збивається на русизми. Крім того, токенизатор ділить кириличні слова на більше частин, і малій моделі бракує параметрів, щоб добре засвоїти граматику малопредставленої мови.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: за temperature більше нуля кожен наступний токен обирається випадково з розподілу ймовірностей. Одна відмінність на початку відповіді змінює весь подальший контекст, тому прогони можуть розійтися від правильної відповіді до помилкової.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Відповідь: галюцинація – це впевнено сформульована, але вигадана відповідь. Модель генерує найімовірніше продовження тексту й не має механізму перевірити,

чи знає факт, а в навчальних даних відповіді «не знаю» трапляються рідко, тому правдоподібна вигадка для неї ймовірніша за відмову.

Висновок за завданням 5

Оцінка за рубрикою дала відтворюваний результат: хмарна модель – 11 балів, qwen3:4b – 9, llama3.2:3b – 3. Найпоказовішими виявилися завдання на формат, стиснення з числами та логіку: вони перевіряються об’єктивно й найсильніше розділяють моделі. Вимогу «без markdown» виконала лише qwen3:4b, тож формат треба забезпечувати програмно. Для роботи з українськомовним текстом я обрав би хмарну модель, а з локальних – qwen3:4b.

Завдання 6. Керування випадковістю: temperature і top_p

Модель – llama3.2:3b (локально). Творче завдання: «Назва освітнього проєкту з астрономії». Фактичне питання: «Рік підписання Паризької кліматичної угоди», правильна відповідь – 2016 (джерело: UNFCCC, https://unfccc.int/sites/default/files/resource/parisagreement_publication.pdf). Кожну комбінацію параметрів запущено 5 разів.

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	4	—
temperature	1.5	5	1
top_p	0.1	1	—
top_p	0.5	3	—
top_p	1.0	4	—

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

0 (або близько до нуля): агенту потрібна передбачувана відповідь, а випадковість збільшує ризик зламаного формату чи вигаданого значення.

2. Яке – генератору варіантів заголовків?

приблизно 0,7–1,0: потрібна різноманітність, а за 0 модель щоразу видає той самий варіант; за 1,5 зростає частка безглузвих назв.

3. Чим вплив top_r відрізняється від впливу temperature за характером, а не за силою?

temperature змінює форму всього розподілу, тож малоімовірні варіанти стають імовірнішими й можуть з'явитися дивні відповіді. top_r не змінює співвідношення ймовірностей, а відкидає «хвіст», тому за малого top_r варіанти обираються лише з кількох найімовірніших.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

У нашому експерименті всі 10 відповідей збіглися («2015»), але повної гарантії temperature=0 не дає через особливості обчислень з плаваючою комою й паралелізацію. Водночас стабільна відповідь виявилась неправильною: модель відтворювано плутає рік ухвалення угоди (2015) з роком підписання (2016).

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

«ЗіркоКаструля: космос для всієї родини на кухні». За високої температури розподіл сплющується, і модель обирає малоімовірні токени; одне невдале слово змінює контекст, і далі текст продовжує цей випадковий напрямок.

Відповіді на питання (крок 8)

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Відповідь: логіти (оцінки токенів) діляться на T перед функцією `softmax`: $p = \exp(z/T) / \sum \exp(z/T)$. За $T < 1$ розподіл стає гострішим і найімовірніший токен домінує, за $T > 1$ – пласкішим, а за $T \rightarrow 0$ завжди обирається найімовірніший токен.

2. Чим `top_p` відрізняється від `temperature` – обидва впливають на різноманітність, але за яким механізмом?

Відповідь: `top_p` (nucleus sampling) сортує токени за ймовірністю, залишає найменший набір, сумарна ймовірність якого досягає p , і вибирає лише серед нього. `Temperature` змінює самі ймовірності, а `top_p` обмежує коло кандидатів, не змінюючи їхнього співвідношення.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: зазвичай рекомендують змінювати один параметр, а другий лишати за замовчуванням, бо їхній спільний ефект важко передбачити й інтерпретувати. На практиці частіше налаштовують `temperature`, залишаючи `top_p = 1`.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: обчислення з плаваючою комою залежать від порядку операцій, який змінюється через паралелізацію та об'єднання запитів у пакети. Коли два токени мають майже однакову ймовірність, мікроскопічна різниця змінює вибір, і далі відповідь розходиться.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: для витягу даних, JSON і коду – 0–0,2, щоб результат був стабільним. Для діалогу – близько 0,5–0,8, для творчих текстів і генерації ідей – 0,8–1,2.

Висновок за завданням 6

Температура регулює випадковість, а не «розумність»: за 0.0 творче завдання дало 1 унікальну назву з 5, за 1.5 – 5, але з'явилися безглузді варіанти. На фактичному питанні висока температура не допомогла (1 правильна з 5), а за 0.0 модель стабільно повторювала неправильний 2015 рік. Для лабораторних зі структурованими даними я використовуватиму temperature = 0 і top_p = 1, бо налаштування за замовчуванням (близько 0,7–0,8) додають випадковість, яка ламає формат.

Підсумкове порівняння моделей

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (gemini-3.5-flash-lite)
Швидкість генерації	≈21,6 ток./с; ≈60 симв./с	≈14,8 ток./с	до 468,3 симв./с
TTFT	≈0,4–0,5 с (холодний старт – 5,9 с)	Не вимірювався окремо; видима відповідь з'являється лише після роздумів	≈0,7–0,8 с
Якість, балів /12	3	9	11
Стабільність формату (завдання 2)	1 бал: markdown-обгортка в обох прогонах	2 бали: чистий JSON	1 бал: markdown-обгортка в обох прогонах

Потрібні ресурси	≈2 ГБ на диску, ≈4 ГБ RAM, працює на CPU	≈2,5 ГБ на диску, ≈5–6 ГБ RAM, працює на CPU	Лише інтернет і API-ключ
Вартість	Безкоштовно	Безкоштовно	Безкоштовний тариф з лімітами, понад ліміт – оплата за токени
Куди йдуть ваші дані	Залишаються на комп'ютері	Залишаються на комп'ютері	На сервери Google
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Прості задачі з наданим текстом, конфіденційні дані, робота без інтернету, коли якість української не критична	Задачі на обчислення й міркування з конфіденційними даними, коли правильність важливіша за час відповіді	Факти, якісна українська, складні відповіді, коли дані не конфіденційні й є стабільний інтернет

Правило вибору: локальну модель доцільно брати, коли дані не можна передавати назовні, потрібна робота без інтернету або без лімітів запитів, а задача зводиться до роботи з наданим текстом. Хмарну – коли важливі точність фактів, якість української та складні міркування, а дані можна відправити сторонньому провайдеру.

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.